

R.A.V.E.N

CLI assistant. Currently uses Openrouter's API; swap in a local model later by writing a new class in `raven/llm_provider.py` that implements `LLMProvider.reply()` and using it in `cli.py`.

Setup

```
pip install -r requirements.txt
export OPENROUTER_API_KEY=your-key-here
python -m raven.cli
```

Headless mode

```
raven -p "What is 2+2?"          # runs one request, prints the plain-
text reply, exits
raven -p "Delete old.log" -y     # -y auto-approves confirmation-gated
tool calls
```

No TTY needed — for scripting, cron, or piping into other tools. Without `-y`, anything that would need your confirmation (`delete_file`, `run_command`, etc.) is denied by default, never silently approved just because no one's watching; the reply says so, and R.A.V.E.N exits normally. On an actual error, it prints the error and exits with status 1. History, settings, and `RAVEN.md` rules all work the same as interactive mode.

Tests

```
pip install -r requirements-dev.txt
pytest
```

Runs in a few seconds, no network and no real model or account touched by default. Everything that needs those (a live OpenRouter call, a real Gmail account) is marked `live` and skipped unless you ask for it explicitly:

```
pytest -m live
```

Project/user rules

Drop a `RAVEN.md` in a project's root and R.A.V.E.N will load it into the system prompt on startup — coding standards, how to run tests, whatever the model should know about that project specifically. A `~/ .raven/RAVEN.md` does the same thing globally, across every project. Both are optional; if present, R.A.V.E.N says so on startup ("Loaded rules from ..."). User rules come first, project rules last — closer to the actual task, so they win on anything the two disagree on. Each file is capped at ~4000 characters (same truncation convention as everything else in this project).

Code editing and search

For working in a codebase: `tree` shows a directory's structure recursively (bounded depth/entry count, so it stays usable on a big project); `glob_files` finds files by name pattern (`**/*.py`); `grep` searches file contents by regex; `edit_file` makes a targeted change to part of an existing file instead of rewriting the whole thing with `write_file` . `edit_file` requires the text being replaced (`old_string`) to match **exactly once** in the file — no match, or more than one, and it refuses rather than guess which occurrence was meant; add more surrounding context to make it unique. Unlike `write_file` (which only confirms when overwriting something that already exists), `edit_file` always confirms, since it only ever touches an existing file — the confirmation shows exactly what's being replaced and with what.

`run_tests` (default command: `pytest`) is a dedicated tool for running a project's actual test suite — same confirmation policy as `run_command` , but with a 120s timeout instead of 30s, since a real test suite (this project's own included) can legitimately take longer than a quick shell command.

Permissions

By default, `run_command` / `run_tests` confirm unless the command is a plain call to a built-in read-only program (`ls` , `cat` , etc.), and `git` confirms unless the subcommand is `status` / `log` / `diff` / `show` .

```
~/ .raven/settings.json
```

lets you extend that trust — additively, never in place of it:

```
{
  "permissions": {
    "allow_commands": ["pytest"],
    "allow_git": ["commit", "add"]
  }
}
```

`allow_commands` lets a plain, metacharacter-free call to that program skip confirmation too (via `run_command` or `run_tests`); `allow_git` does the same for a git subcommand. The shell-metacharacter check always still applies regardless: `"pytest; rm -rf ~"` still confirms even with `pytest` trusted — trusting a program name only ever exempts a plain call to exactly that program, never anything chained after it.

Hooks

`~/ .raven/settings.json` can also run a shell command automatically before and/or after a specific tool call — e.g. auto-formatting a file right after R.A.V.E.N edits it:

```
{
  "hooks": {
    "post": {"edit_file": "black {path}"},
    "pre": {"edit_file": "cp {path} {path}.bak"}
  }
}
```

`{placeholder}` s are filled in from that tool call's own arguments (`{path}` , or anything else the tool takes). Hooks never ask for confirmation — putting one in this file, which only you control, **is** the approval, the same trust boundary as `permissions` above. A `post` hook only runs if the tool call actually succeeded (not after `edit_file` 's own "not found" refusal, which changed nothing); a hook's own output or error is appended to the tool's result so you can see it happened. Scoped to the model's own tool-calling loop — manual `/write` , `/run` , etc. don't trigger hooks.

Skills (on-demand tools)

Only file/shell/coding tools (`read_file` , `edit_file` , `grep` , `run_command` , `git` , `save_note` , etc.) are sent to the model on every request. Desktop app control, the web, R.A.V.E.N's own browser, Gmail, and vision are grouped into named **skills** — `desktop` , `web` , `browser` , `gmail` , `vision` — that the model unlocks itself with a `load_skill` call the moment a request actually needs one, then uses like any other tool for the rest of that session. `/skills` shows which are currently loaded. This cut R.A.V.E.N's fixed per-request overhead (system prompt + every tool's spec, sent regardless of what a conversation is actually about) by roughly half — measured at ~5.6k tokens down to ~2.75k when no skill is loaded, since neither the unused tool specs nor their guardrail instructions (e.g. the Gmail Reply-To/untrusted-content guidance, the browser submit-safety rule) are paid for until they're relevant. Nothing about how a loaded skill's tools behave changes — same confirmation policy, same everything — this only affects when their specs and instructions actually reach the model. A fresh session always starts back at core-only; there's no persistence to manage.

Voice input

Press `ctrl+v` at the prompt to start recording, `ctrl+v` again to stop — transcribed locally and offline via [faster-whisper](#) (no OpenRouter quota used, no audio sent anywhere) and inserted into the input buffer for you to review, edit, or discard, exactly like anything you'd typed — it never auto-sends. The bottom toolbar shows `[ctrl+v: voice input]` when ready and `[ctrl+v: recording – press again to stop]` while recording.

Needs two optional Python packages plus, **on Linux specifically**, a system library:

```
pip install sounddevice faster-whisper
sudo apt install libportaudio2 # Debian/Ubuntu and derivatives (Zorin,
Mint, ...)
# Fedora: sudo dnf install portaudio
# Arch:   sudo pacman -S portaudio
```

That last step is a real, unavoidable requirement of the `sounddevice` package on Linux — its wheel doesn't bundle the PortAudio binary there the way it does on Windows/macOS, so without the system library it fails to import with `OSError: PortAudio library not found`. None of this is required to run R.A.V.E.N at all: without it, `ctrl+v` just inserts a "voice input not set up" message instead of recording, and everything else works exactly the same.

One-time model download (explicit, never a side effect of a keypress):

```
python -m raven.voice          # downloads the Whisper "base" model
                               (~140MB), resumable
```

Until that's done, `ctrl+v` just shows "voice model not downloaded — run: `python -m raven.voice`" in the toolbar. If the download makes no progress (0-byte `.incomplete` files under `~/.cache/huggingface/hub`), retry with `HF_HUB_DISABLE_XET=1 python -m raven.voice` — the newer Hugging Face "xet"

transfer path stalls on some networks. Transcription runs in a background thread, so the prompt stays responsive; `ctrl+c` at the prompt clears the line (it doesn't crash), `ctrl+d` on an empty prompt exits. Note `ctrl+v` is bound to voice, so it does not paste — use your terminal's paste shortcut (usually `ctrl+shift+v`).

Vision (image analysis)

`analyze_image` (in the `vision` skill) looks at an image file — a screenshot, a photo, a diagram — and answers a question about it, using a **separate**, configurable model (`settings.json` 's `model.vision`) since the main chat model isn't necessarily vision-capable. `browser_screenshot` (in the `browser` skill) saves a PNG of the current page in R.A.V.E.N's own browser session for `analyze_image` to look at — the two compose rather than being one merged tool, same as `grep` / `glob_files`. No confirmation needed for either (read-only). Capturing a screenshot of your actual desktop (rather than just R.A.V.E.N's own browser page) isn't supported yet — it needs the Wayland/GNOME screenshot portal's more involved async permission flow, a deliberate scope decision, not an oversight.

Structure

```
raven/  
  llm_provider.py    # LLM interface + Provider (Openrouter)  
implementation, RAVEN.md loading  
  assistant.py      # conversation state, calls the provider  
  tools.py          # file I/O + shell/search primitives, tool specs  
  browser.py        # Playwright-backed browser session (own thread)  
  gmail.py          # Gmail OAuth, list/read/send/reply  
  store.py          # SQLite persistence for conversation history  
  statusline.py     # bottom status-line segments  
  config.py         # env/config loading + settings.json  
  cli.py            # entry point, input/output loop  
tests/              # pytest suite – see "Tests" above
```

Memory

Conversation history is persisted to `~/.raven/history.db` (SQLite) and reloaded on startup, so context survives between runs.

This can go stale: if system state changes between turns (an app gets installed, a window closes) an old tool result sitting in history can get treated as still true. R.A.V.E.N is told to re-check rather than trust a stale "X isn't available" from earlier in the conversation, but if it still seems to be working from old information, run `/forget` to clear history (in memory and on disk) and start fresh.

Only a bounded window of recent history is actually sent to the model each request (see "Status line" below for the budget). Once a conversation grows past that, older turns aren't just dropped — they're summarized into a running summary that's sent in their place, so facts and decisions from earlier in a long conversation are still available, just compressed. This happens automatically whenever it's needed, or on demand with `/compact`. The summary itself is never written to disk — the full raw conversation in `~/.raven/history.db` is untouched either way, so nothing is ever actually lost, and a fresh session always starts from the real thing.

Separately, R.A.V.E.N can save short, durable facts to `~/.raven/memory/notes.md` with its own `save_note` tool — a stated preference, a project convention, a correction — loaded into every future

session automatically, so they survive `/forget` and outlive any single conversation. It decides on its own when something's worth remembering; it never needs confirmation, since it's an append-only, bounded write (one note capped at ~500 characters, the whole file at ~4000 — past that it refuses rather than silently truncating a fact or evicting an older one). `/notes` shows what's currently saved; the file is a plain markdown list you can edit or clear by hand at any time.

Status line

The prompt shows a bottom status line. Segments and their order come from `~/.raven/settings.json` (optional):

```
{
  "model": {
    "name": "nvidia/nemotron-3.5-lightning:free",
    "fallbacks": ["nex-agi/nex-n2.5-pro:free", "openrouter/free"]
  },
  "statusline": { "segments": ["model", "context", "tokens", "memory", "cpu"] }
}
```

Status-line segments: `model` (what actually answered), `context` (approx. tokens of history sent to the model / the budget), `tokens` (session total), `memory`, `cpu`. Add your own in `raven/statusline.py` — one function per segment.

Change it from inside R.A.V.E.N with `/statusline` (no args lists the segments; `/statusline model tokens cpu` sets and saves them).

`model.fallbacks` are tried in order when the primary free model is down or rate-limited (OpenRouter's `models` routing).

Opening applications (Linux)

Ask in natural language ("open Firefox", "launch the file manager") and R.A.V.E.N resolves the name against your installed `.desktop` entries — covering native packages, Flatpak, and Snap installs — and launches it with `gtk-launch` — no confirmation prompt, since it's not a destructive action. Ambiguous or unknown names get you a candidate list instead of a guess.

To open a specific *file*, R.A.V.E.N uses `open_file` : with no app given it opens with the system default handler (`xdg-open`); with an app name, it matches it the same way as `open_app` and passes the file to `gtk-launch` as the file/URI argument. Ask it to create a file and open it, and it just calls `write_file` then `open_file` .

App-name matching is whole-word, both directions ("text editor" matches "the text editor app" and vice versa) so generic phrasing works without guessing a brand. If nothing matches, it's offered `list_apps` — every installed app's display name — instead of picking a plausible-sounding one from general knowledge.

`close_app` closes a running app by signalling its process — this closes **every** window of that app at once, and always asks for confirmation first since it can lose unsaved work.

To close just *one* window of a multi-window app (e.g. one of several VS Code or browser windows), use `list_windows` / `close_window` instead. These need the [Window Calls](#) GNOME Shell extension (`window-calls@domandoman.xyz`) — plain X11 tools like `wmctrl` can't see app windows at all on GNOME/Wayland, since they're native Wayland surfaces with no X11 proxy. Install it, then:

```
gnome-extensions install window-calls@domandoman.xyz.shell-extension.zip
# log out and back in – GNOME Shell only rescans extensions on Wayland at login
gnome-extensions enable window-calls@domandoman.xyz
```

`close_window` matches a window by a substring of its title (see `list_windows` for exact titles) and closes only that one, gracefully (Mutter's own close, same as clicking the × button) — the other windows of the same app are untouched.

Web access

- `open_url` — opens a URL in the default browser, or a specific named one (e.g. "open figma.com in Firefox" when Firefox isn't your default).
- `fetch_url` — fetches a page and returns its text (HTML stripped, truncated).

- `web_search` — searches the web via DuckDuckGo's HTML results (no API key needed). This is a free, keyless, and therefore rate-limited endpoint — it can start returning a "temporarily rate-limited" message after a burst of searches. There's no paid fallback configured; if that's too flaky in practice, the fix is a real search API key, not more scraping tricks.

Browser interaction

For pages that need JavaScript, or that R.A.V.E.N needs to click/type on, `browser_navigate` / `browser_read` / `browser_click` / `browser_type` / `browser_submit` / `browser_close` drive a real, persistent browser session via [Playwright](#) — `pip install playwright` plus a one-time `playwright install chromium` (downloads a ~115MB browser binary).

- **Isolated:** a fresh Chromium session with none of your real browser's cookies or logins — it can read and interact with public pages, but can't act as you on sites you're signed into elsewhere.
- **Headless:** runs invisibly, no window pops up on your desktop. Its status-line entries and replies are how you see what it's doing.
- **Text-based selectors:** click/type target visible text or labels, not CSS selectors, since the model can act on what it reads without needing raw HTML.
- **Submitting requires confirmation, and it's enforced, not just asked for:** `browser_click` inspects the DOM before clicking — a real HTML submit control (a `<button>` / `<input type=submit>`, or a bare `<button>` inside a `<form>`) gets refused and redirected to `browser_submit`, which goes through the same confirmation gate as `run_command` / `delete_file`. This is a structural check, not model judgment — but it only covers real HTML submit semantics. A JS-driven button that commits an action without being a literal submit control (e.g. a script-handled "Buy Now") isn't caught by it; R.A.V.E.N is instructed to use `browser_submit` itself for anything that commits, but that part relies on the model getting it right, same as other soft boundaries in this project (`open_app` vs `open_file`).

Gmail access

`gmail_list_messages` / `gmail_read_message` let R.A.V.E.N read your real Gmail. `gmail_send_message` sends a **new** email as you, and `gmail_reply_message` replies to an existing one — in its own thread, to whoever the original says to reply to (its `Reply-To`, else its `From`). Both need **your confirmation** each time, and the prompt shows the full To/Subject/Body, the way `run_command` shows the full command.

For a reply the recipient is worked out from the original message, so the prompt shows the *resolved* address rather than the visible sender. That matters: a sender can set `Reply-To` to point replies somewhere else, and you should see that before approving. It's plain reply only — no reply-all.

The OAuth scopes granted are exactly `gmail.readonly` + `gmail.send` — deliberately not `gmail.modify`, so even with a code bug, Google's API itself refuses to delete or alter existing mail. (Replying needs no extra scope: a reply is just a send carrying a thread id and `In-Reply-To` / `References` headers, so it's *our* tool boundary, not Google's, that limits R.A.V.E.N to plain replies.) A recipient containing a line break is refused outright, since that could smuggle an extra recipient in.

This needs a one-time setup only you can do (it requires your Google account and browser-based consent):

1. Go to the [Google Cloud Console](#), create a project (or use an existing one).
2. Enable the **Gmail API** for it (APIs & Services → Library → search "Gmail API" → Enable).
3. Configure the **OAuth consent screen** (APIs & Services → OAuth consent screen). "External" + "Testing" mode is fine for personal use — add your own Google account under "Test users".
4. Create credentials: APIs & Services → Credentials → Create Credentials → OAuth client ID → Application type **Desktop app**.
5. Download the resulting JSON and save it as `~/.raven/gmail_credentials.json`.
6. Ask R.A.V.E.N to do something with Gmail. The first call opens a browser for you to log in and grant access (read + send); after that, a cached token at `~/.raven/gmail_token.json` covers every run until it's revoked or expires — no repeated logins.

Until step 5 is done, Gmail tools return a clear "not set up yet" message instead of an error or a guess. If you'd already completed setup before `gmail_send_message` was added, the cached token only has the read scope — delete `~/.raven/gmail_token.json` and go through step 6 again to re-grant with send included.

Swapping to a local model later

Add e.g. `OllamaProvider(LLMProvider)` in `llm_provider.py`, then in `cli.py` change one line: `provider = OllamaProvider(...)`. Nothing else in the codebase changes.